ZeroID — Engineering Status Report

Prepared for: External strategy/architecture consultant (for review & opinion) **Date:** 28 June 2026 (rev. 2 — adds AI Agent Passport v1, §5.4) **Subject:** ZeroID re-platforming onto the canonical Aethelred protocol; implementation of the institutional ("20×") moat features; and a new "AI agent's passport" (AI Agent Identity v1) vertical **Status of work:** Implemented and tested on a feature branch; activation gated on testnet. Not yet merged or deployed.

---

1. Executive summary

ZeroID has been **re-based onto the real canonical Aethelred stack** and the **institutional moat features from your strategy documents have been implemented and tested in code**. Importantly, this was a *conformance* exercise, not a rewrite: ZeroID's existing cryptography (Circom/Groth16 over BN254) and EVM/Solidity contracts were already compatible with the protocol's on-chain verifier, so the work was to make ZeroID **consume** the protocol's canonical capabilities instead of its own parallel re-implementations.

All work is consolidated behind a single, CI-enforced integration seam (`src/lib/aethelred/`), which doubles as the **reusable conformance template** for the other four ecosystem dApps (Cruzible, TerraQura, NoblePay, Shiora) — directly addressing the "no coordination between teams" problem.

Headline status: 108 test suites / 2,367 unit tests + 6 Solidity (Foundry) tests passing; type-check clean; boundary guard enforced in CI. Most canonical paths are built and **flag-gated OFF** so nothing breaks pre-testnet; activation is configuration (env flips), not code, with zero regression risk.

AI Agent Passport v1 (new — see §5.4): Per your *AI Agent Identity v1* spec, ZeroID is now positioned as **"the AI agent's passport."** The flagship vertical — an AI agent requesting an eligibility proof on behalf of its KYC'd controller, tightly scoped and fully audited — is implemented end-to-end (backend policy + service + route that reuses the *exact* human eligibility logic; frontend register wizard + live list + audit attribution). It lives on its own branch (`feat/ai-agent-passport-v1`): **21 backend AI/eligibility tests + the full frontend suite (110 suites / 2,372) green**, type-check clean. Maturity is honestly bounded to "Pilot / Preview" pending the database migration.

The one item that most needs your opinion is in §2 (a correction to an assumption in your technical specs), followed by the open questions in §10.

---

2. Critical finding — canonical stack reconciliation (please weigh in)

Your three documents assumed the protocol's proving layer is **Halo2 / PlonKish** with a **Cosmos-SDK Go + Rust-FFI** verifier. We verified the assumption against the canonical Aethelred monorepo (the source-of-truth repo, governance policy adopted 2026-04-05). The reality:

Dimension	Your spec assumed	Canonical Aethelred (verified)	Verdict
Chain	Cosmos-SDK / CometBFT (Go)	Cosmos-SDK v0.50 / CometBFT v0.38 (Go 1.24)	Correct
Verifier	Go module + Rust FFI	Go modules + custom **Rust VM** (`crates/vm`) with ZK precompiles	Correct
General ZK system	**Halo2 / PlonKish**	**Groth16 + PLONK over BN254/BLS12-381** (arkworks)	Differs

Dimension	Your spec assumed	Canonical Aethelred (verified)	Verdict
zkML	EZKL / Halo2	**EZKL** (Halo2-backed) precompile + Freivalds for LLM inference	Correct (EZKL is the zkML lane)
TEE	Decentralized TEEs / SGX-SEV	**Real DCAP attestation, hardware-agnostic, 6 platforms**	Correct
Crypto	(PII purge, ZKP)	**Post-quantum ML-DSA-65 + ECDSA hybrid, ML-KEM-768**, BLS12-381	Correct + stronger
On-chain cost	~210k gas flat	precompile-based, flat-cost design	Consistent

Implication: the canonical general-purpose ZK system is **Groth16/PLONK** (BN254); **EZKL (which is Halo2-backed) is specifically the zkML lane.** This is *good news* — ZeroID's existing Groth16/BN254 attribute circuits map directly to the chain's Groth16 precompile, so no proving-system migration was required.

Question for you: do you concur with treating **Groth16/PLONK for attribute proofs + EZKL for zkML liveness** as the canonical split? And is your zkML spec's byte-format guidance (G2 Fp2 limb order, point compression) something you can confirm against a known proof, so we can finalise the snarkjs→arkworks serialisation (see §9, gate W2c)?

---

3. Starting point

ZeroID was a strong, genuinely-built self-sovereign identity platform (Next.js 14, Express/Prisma, 12 Solidity contracts, 9 Circom/Groth16 circuits, a software-simulated TEE crate, Go/Python SDKs) — but built **in parallel** to the protocol: its own chain client, its own ZK verifier contract, a simulated TEE, ECDSA-only signing, and bespoke hooks. It had even been de-vendored ("retired") from the protocol monorepo during the move to a hub-and-spoke (standalone-repos) model — a process change, not a quality judgement.

---

4. Strategy adopted

Conformance-first via a strangler-fig boundary. A single internal module, `src/lib/aethelred/`, wraps the canonical SDK and chain. ZeroID's chain access was migrated subsystem-by-subsystem behind this seam; each parallel implementation is retained as a flag-gated fallback until a live-node check lets us delete it. The app keeps running throughout, and every increment is independently shippable. A CI guard forbids any other file from importing the SDK — which is what stops re-divergence and makes the boundary a copyable standard for the other dApps.

The deeper rationale: the protocol owns the hard primitives (DCAP attestation, EZKL, ML-DSA-65, Digital Seals), so the moat features **compose** canonical capabilities rather than reinventing weaker versions — exactly the structural advantage your positioning report describes.

---

5. Work completed

5.1 Phase 1 — Conformance (workstreams W1–W6)

WS	Delivered	Status
W1	Conformance boundary client + `@aethelred/sdk` wiring; **chain-ID fix 7331/7332 → canonical 8821/88210**	tested
W2	ZK verification routed to the chain precompile (Groth16/PLONK/EZKL); canonical wire encoding; verifying-key registry; flag-gated strategy	tested (activation gated)
W3	TEE attestation → canonical **DCAP** verify; **Digital Seals** (`x/seal`) adapter	tested (activation gated)
W4	**ML-DSA-65 + ECDSA** post-quantum hybrid signing	tested (activation gated)
W5	Canonical `@aethelred/sdk/react` hooks (seal/job state)	tested
W6	Ecosystem compatibility CI: boundary import guard + GitHub Actions workflow (builds against pinned protocol SHA)	enforced

5.2 Phase 2 — Institutional moat

- **EZKL zkML liveness** (`liveness.ts`): verifies a zero-knowledge liveness inference on the chain's EZKL rail, and a combined check requiring **both** the zkML proof **and** a DCAP hardware attestation (`VerificationResult{zk_proof_verified, tee_attestation_verified}`). unit-tested. (The proof-*production* pipeline — model → ONNX → EZKL circuit → keys — is gated on the EZKL toolchain; see §9.)
- **Conditional disclosure / key-split escrow** — the FATF travel-rule "asymmetric key-split escrow", complete across both layers:
  - Off-chain (`shamir.ts`, `disclosure.ts`): AES-256-GCM-encrypt the payload, **Shamir t-of-n split** the key to a compliance quorum, anchor only `sha256(ciphertext)` (zero PII), reconstitute under quorum, and **key-shred erasure** (destroying shares makes the on-chain commitment permanently un-linkable — GDPR/DPR-2021 on an immutable ledger). real crypto round-trip tested.
  - On-chain (`contracts/ConditionalDisclosure.sol`): anchors the commitment + an un-linkable nullifier, gates disclosure behind a **t-of-n compliance-officer quorum bound to a warrant hash**, supports erasure. 6 Foundry tests.
  - TS client + one-call orchestrator (`disclosure-contract.ts`, `disclose.ts`). tested.

5.3 Protocol-side fix

The canonical SDK had a complete ML-DSA-65 implementation that **was never exported** from its public entry point, so no dApp could use it. We exported it (Aethelred branch `feat/sdk-export-pqc`) — a one-line fix that unblocks post-quantum signing **for all five dApps**.

5.4 AI Agent Passport v1 — "the AI agent's passport"

Implementing your *AI Agent Identity v1* spec. We grounded it in the real codebase first: the agent registry already existed (`AgentIdentityService`), so v1 is the **constrained agent→eligibility vertical** layered on additively — not new infrastructure, and with no destabilisation of the human workflow.

- **Scopes + policy (pure, unit-tested):** a controlled read-only vocabulary (`eligibility.read` / `audit.read` / `identity.read`) and `POLICY_AGENT_ELIGIBILITY_VIEW_V1` — an agent may act only if its

own credential is in good standing AND scoped AND the controller is itself eligible AND the agent's risk ceiling covers the controller's tier (the *layered trust object* from your spec), with your exact deny-codes.

- **Agent→eligibility wrapper (dependency-injected):** scope + policy check → delegate → record an `AgentAction`. Returns your `AgentEligibilityProofResponse` (actor + proof + evaluation + evidence incl. `agentActionId`).
- `POST /api/v1/ai/agents/eligibility/proof` with your full error contract (400/401/403/404/422/500).
- **Eligibility reuse — the integration seam, now closed:** the human eligibility logic was *inline* in `routes/verification.ts` (no callable service). We extracted it **mechanically** into an exported `eligibilityProofHandler` (byte-identical; the six pre-existing eligibility route tests confirm parity), and the agent path reuses it **in-process via a response shim — no re-implementation, the human workflow untouched.**
- **Additive schema:** `scopes`, `maxRiskTier`, `controllerDid`, `policyId`, `decision` (+`AgentActionDecision`). `prisma generate` done; the **migration is the only DB-gated step.**
- **Frontend:** v1 API client + `useAIAgents` / `useCreateAIAgent` hooks; the agent-identity **register wizard now performs real registration** (name + scope multi-select + max risk tier) behind a "Pilot / Preview" banner with a live agent list; the **audit timeline has an "Agent actions" filter** rendering *"AI Agent X acting for Controller Y."*

Maturity is bounded to **"Pilot / Preview"** pending the migration, matching your phased rollout (schema+backend → internal UI → pilot external agents). Policy details: `docs/policies/agent_identity_v1.md`.

---

6. Mapping to your "20× moat" pillars

Your pillar (Strategic Positioning Report)	Implemented as	Status
Hardware isolation (TEE)	DCAP attestation adapter (`attestation.ts`)	Built; activation needs raw DCAP quote (W3c)
Verifiable AI (zkML)	EZKL liveness verify, zkML + DCAP (`liveness.ts`)	Built; needs trained model/circuit (Phase 2b)
Zero-PII / right-to-be-forgotten	nullifier + commitment-only + key-shred erasure	Built & tested
Conditional jurisdictional disclosure (FATF)	Shamir key-split + on-chain compliance quorum + warrant	Built & tested (off-chain + on-chain)
Quantum-resistant sovereign identity	ML-DSA-65 + ECDSA hybrid signing	Built; needs backend injection (W4c)
Continuous verification	Digital Seals + (periodic) re-attestation	Seals built; cadence is a product decision
Economic flywheel ($0.10 / 50% burn / 50% Cruzible)	—	**Not built — deferred** (cross-app; see §9)

---

7. Architecture (the boundary)

`src/lib/aethelred/` — 15 modules, fully documented in its own README + activation runbook: `client` · `zk`+`encoding`+`vkeys`+`verify` (ZK) · `attestation` (DCAP) · `seals` · `signing` (PQC) · `react` (hooks) · `liveness` (zkML) · `shamir`+`disclosure`+`disclosure-contract`+`disclose` (conditional disclosure). Two planes: the canonical **Cosmos-REST** plane (SDK) for verification/seals/attestation, and the **EVM** plane (viem) for Solidity contracts — ZeroID already ran on the protocol's EVM RPC.

---

8. Verification & evidence

- **108 unit-test suites / 2,367 tests** passing (clean tree), incl. 15 boundary suites / 59 boundary tests.

- **6 Solidity (Foundry) tests** for the on-chain disclosure quorum; project compiles (solc 0.8.28).
- TypeScript `tsc --noEmit` clean; **boundary import guard** passing (240 files scanned); GitHub Actions compatibility workflow added.
- Developed test-first (TDD) throughout; the off-chain crypto and the Solidity contract are tested for real, while chain-integration paths are unit-tested with mocks pending the live node.
- **Branches (not merged/pushed):** ZeroID `design/aethelred-conformance-moat` (20 commits, conformance + moat); ZeroID `feat/ai-agent-passport-v1` (6 AI Agent Passport commits atop that branch); Aethelred `feat/sdk-export-pqc` (1 commit). Full commit lists in the appendix.
- **AI Agent Passport v1:** 21 backend AI/eligibility tests + full frontend suite (110 suites / 2,372 tests) green; type-check clean. The eligibility extraction is parity-verified by the 6 pre-existing eligibility route tests.

---

9. Current maturity & activation gates (honest framing)

ZeroID is best described as **"testnet-candidate: conformance complete, moat cores implemented and tested, activation pending the live testnet."** The remaining work is deliberately *not* code we could responsibly fabricate without the running chain:

Gate	What it unlocks	Needs
W2c	Canonical ZK verification live	Live-node proof byte-format confirmation + on-chain verifying-key registration → flip `NEXT_PUBLIC_CANONICAL_VERIFY` + `NEXT_PUBLIC_AETHELRED_VKEYS`
W3c	DCAP attestation live	TEE worker surfaces the raw DCAP quote/pcrValues
W4c	PQC signing live	Inject a real ML-DSA-65 backend via `configurePQCProvider`, flip `NEXT_PUBLIC_PQC_SIGNING`
Phase 2b	zkML liveness live	Train liveness CNN → ONNX → EZKL circuit/keys → register Circuit on-chain
Deploy	On-chain conditional disclosure live	Deploy `ConditionalDisclosure.sol`, grant roles, wire contract address
Tokenomics	Economic flywheel	Product/protocol decision (see open questions)

Each activation is a config flip with no code change and no regression window — by design of the strangler-fig approach.

---

10. Open questions for your opinion

1. **Proving split** — confirm Groth16/PLONK (attributes) + EZKL/Halo2 (zkML) is the right canonical model, and the snarkjs→arkworks byte-format (G2 limb order/compression).
2. **Conditional disclosure design** — is "AES-GCM payload + Shamir t-of-n split of the symmetric key + sha256 commitment anchored as a Digital Seal + on-chain warrant-bound quorum + key-shred erasure" sufficient for the FATF Travel Rule and ADGM DPR-2021/GDPR as you envisaged the "asymmetric key-split escrow"? Would you prefer threshold/MPC encryption over Shamir-of-symmetric-key for the quorum?
3. **Liveness** — agree that the strong claim should require **both** the zkML proof and a DCAP attestation? And what model size/latency is realistic for an ADFW demo — your spec's ~2.8s edge proof of a 15M-parameter CNN looks aggressive for current EZKL; should we plan a smaller model or a TEE-side fallback?

4. **Post-quantum** — is ML-DSA-65 hybrid signing sufficient, or do you also want ML-KEM-768 key-encapsulation surfaced for confidential channels?
5. **Tokenomics** — is the **$0.10 / 50% burn / 50% Cruzible** flywheel still the plan, and should it live in ZeroID or at the protocol level? (Currently unbuilt; it is the main investor-narrative piece still open.)
6. **ADFW 2027 positioning** — given the testnet-gated maturity, how would you frame "implemented, tested, activating on testnet" to institutional investors without overstating live deployment?
7. **AI Agent Passport (v1 auth + evidence)** — the agent path currently authenticates with the controller's JWT (the controller acts on the agent's behalf). Do you want a dedicated agent-token credential signed by the agent identity for v1, or is controller-auth acceptable for the pilot? And should agent-initiated `AgentAction`s also be anchored as on-chain Digital Seals, or is the off-chain action ledger sufficient for v1?

---

11. Recommended next steps

1. Review the two branches (self-contained, documented).
2. Stand up the testnet; flip the activation gates one verified circuit/flag at a time (runbook in `src/lib/aethelred/README.md`).
3. Replicate the boundary to the other four dApps (4-step guide in the README).
4. Decide the tokenomics design and owner.
5. Run the AI Agent Passport schema migration on a test database (`prisma migrate`), then enable agent registration for an internal "Compliance Copilot" pilot.

---

12. Appendix

ZeroID branch `design/aethelred-conformance-moat` — commits (newest first):

```
49b9731 docs(aethelred): boundary architecture + activation runbook
81fc13a feat(aethelred): end-to-end conditional disclosure orchestrator
9c7f902 feat(aethelred): ConditionalDisclosure TS client (viem wiring)
4d9a1dd feat(contracts): ConditionalDisclosure on-chain compliance-quorum gate
e5648e3 feat(aethelred): conditional disclosure key-split escrow (Shamir + Seals)
21bbced feat(aethelred): EZKL zkML liveness verification + TEE binding (Phase 2a)
350e9c2 feat(aethelred): ecosystem compatibility CI (boundary guard + workflow)
2d8bfcb feat(aethelred): canonical react hooks (client-injected useSeal/useJob)
312a4f7 feat(aethelred): PQC hybrid signing adapter (ML-DSA-65 + ECDSA)
a45b6e5 feat(aethelred): Digital Seals boundary adapter (x/seal)
871f62f feat(aethelred): TEE attestation boundary adapter (DCAP verify + platform map)
e520d89 feat(aethelred): prefer canonical verification at call sites (flag-gated)
450eb43 feat(aethelred): ZKProof -> canonical verify adapter
3fd1f13 feat(aethelred): canonical ZK wire encoding (field elements, proof bytes)
da8dfcb feat(aethelred): route ZK proof verification through the conformance boundary
ce0f98e fix(config): reconcile chain IDs to canonical 8821/88210 (ecosystem manifest)
3ff25a9 feat(aethelred): conformance boundary client (verification + seals modules)
0ee5bc0 feat(aethelred): wire @aethelred/sdk dependency (local file link)
e7fef3a docs: W1 conformance-boundary implementation plan (TDD)
659ca62 docs: ZeroID x Aethelred canonical conformance & 20x moat design spec
```

Aethelred branch `feat/sdk-export-pqc`: `f5a408ae feat(sdk-ts): export PQC (ML-DSA/ML-KEM) from crypto entrypoint`

ZeroID branch `feat/ai-agent-passport-v1` — AI Agent Passport commits (atop the conformance branch, newest first):

```
09866e6 feat(ai): audit timeline agent-actions filter + attribution
d331444 feat(ai): wire register wizard to useCreateAIAgent (real v1 registration)
c703584 feat(ai): close eligibility integration seam - agent reuses human handler
```

```
4baf980 feat(ai): AI Agent Passport v1 frontend - client + hooks + page wiring
b7c19e2 feat(ai): agent eligibility route + additive schema + policy doc
f13684b feat(ai): AI Agent Passport v1 core - scopes + policy + eligibility wrapper
```

AI Agent Passport policy doc: `docs/policies/agent_identity_v1.md`.

Design specs & plans: `docs/superpowers/specs/` and `docs/superpowers/plans/` (design spec, per-workstream TDD plans, Phase 2 zkML-liveness and conditional-disclosure designs).

Reproduce the verification:

```
npx jest --silent                                          # 2,367 unit tests
forge test --match-contract ConditionalDisclosureTest  # 6 Foundry tests
npm run type-check && npm run boundary:check            # types + boundary guard
```